COMPUTER ENGINEERING

QUEZON CITY

| | | | |
|---|---|---|---|
| **COURSE CODE/TITLE:** | CPE 010 - Data Structures and Algorithms | **SECTION:** | CpE21S2 |
| **DATE PERFORMED:** | 9/16/2026 | **DATE SUBMITTED:** | 9/23/2026 |
| **NAME:** | Fernandez, Cedric S. | **INSTRUCTOR:** | Engr. Jimlord M. Quejado |

**Hands-on Activity 9.1 Tree ADT**

**1. PROCEDURE OUTPUT**

**table 9-1**

Main.cpp

```cpp
#include <iostream>
#include "Tree.h"


int main() {

    Node* A = new Node('A');
    Node* B = new Node('B');
    Node* C = new Node('C');
    Node* D = new Node('D');
    Node* E = new Node('E');
    Node* F = new Node('F');
    Node* G = new Node('G');
    Node* H = new Node('H');
    Node* I = new Node('I');
    Node* J = new Node('J');
    Node* K = new Node('K');
    Node* L = new Node('L');
    Node* M = new Node('M');
    Node* N = new Node('N');
    Node* P = new Node('P');
    Node* Q = new Node('Q');

    addChild(A, B);
    addChild(A, C);
    addChild(A, D);
    addChild(A, E);
    addChild(A, F);
    addChild(A, G);
    addChild(D, H);
    addChild(E, I);
    addChild(E, J);
    addChild(J, P);
    addChild(J, Q);
    addChild(F, K);
    addChild(F, L);
    addChild(F, M);
    addChild(G, N);

    return 0;

}
```

Tree.h

```cpp
#ifndef TREE_H
#define TREE_H
#include <iostream>

struct Node {
    char data;

    Node* firstChild;
    Node* nextSibling;

    Node(char value) {
        data = value;
        firstChild = nullptr;
        nextSibling = nullptr;
    }
};

void addChild(Node* parent, Node* child) {

    if (parent->firstChild == nullptr) {
        parent->firstChild = child;
    }
    else {

        Node* current = parent->firstChild;

        while (current->nextSibling != nullptr) {
            current = current->nextSibling;
        }

        current->nextSibling = child;
    }
}
#endif
```

Table 9.2

| NODE | HEIGHT | DEPTH |
|---|---|---|
| A | 3 | 0 |
| B | 0 | 1 |
| C | 0 | 1 |
| D | 1 | 1 |
| E | 2 | 1 |
| F | 1 | 1 |
| G | 1 | 1 |
| H | 0 | 2 |
| I | 0 | 2 |
| J | 1 | 2 |
| K | 0 | 2 |
| L | 0 | 2 |
| M | 0 | 2 |
| N | 0 | 2 |
| P | 0 | 3 |
| Q | 0 | 3 |

Table 9.3

3.1

| | |
|---|---|
| Pre-order | A B C D H E I J P Q F K L M G N |
| Post-Order | B C H D I P Q J E K L M F N G A |
| In-Order | B A C H D I E P J Q K F L M N G |

**Table 9-4**

**Main.cpp**

```
std::cout << "Pre-order: ";
preOrder(A);
std::cout << std::endl;

std::cout << "Post-order: ";
postOrder(A);
std::cout << std::endl;

std::cout << "In-order: ";
inOrder(A);
std::cout << std::endl;
```

**Tree.h**

```
void preOrder(Node* root) {

    if (root == nullptr) {
        return;
    }

    std::cout << root->data << " ";

    Node* child = root->firstChild;

    while (child != nullptr) {
        preOrder(child);
        child = child->nextSibling;
    }
}

void postOrder(Node* root) {

    if (root == nullptr) {
        return;
    }

    Node* child = root->firstChild;

    while (child != nullptr) {
        postOrder(child);
        child = child->nextSibling;
    }

    std::cout << root->data << " ";
}

void inOrder(Node* root) {

    if (root == nullptr) {
        return;
    }

    Node* child = root->firstChild;

    // First child
    if (child != nullptr) {
        inOrder(child);
    }

    // Root
    std::cout << root->data << " ";

    // Remaining children
    if (child != nullptr) {

        child = child->nextSibling;

        while (child != nullptr) {
            inOrder(child);
            child = child->nextSibling;
        }
    }
}
```

**Output**

```
Pre-order: A B C D H E I J P Q F K L M G N
Post-order: B C H D I P Q J E K L M F N G A
In-order: B A C H D I E P J Q K F L M N G
```

**OBSERVATION:** The following shows how each traversal acts when it comes to pre order, post order, and in order also showing that our prediction regarding table 3.1 was correct in every terms that in which pre-order visits the

root first before moving down through the branches, post-order processes all child nodes before visiting their parent, and in-order processes the first child branch, prints the parent, and then visits the rest of the children.

**TABLE 9-5**

**Main.cpp**

```
std::cout << "\nFind D using Pre-order: ";
findData(1, 'D', A);

std::cout << "Find D using Post-order: ";
findData(2, 'D', A);

std::cout << "Find D using In-order: ";
findData(3, 'D', A);
```

Tree.h

```
bool findPreOrder(Node* root, char key) {

    if (root == nullptr) {
        return false;
    }

    if (root->data == key) {
        return true;
    }

    Node* child = root->firstChild;

    while (child != nullptr) {

        if (findPreOrder(child, key)) {
            return true;
        }

        child = child->nextSibling;
    }

    return false;
}

bool findPostOrder(Node* root, char key) {

    if (root == nullptr) {
        return false;
    }

    Node* child = root->firstChild;

    while (child != nullptr) {

        if (findPostOrder(child, key)) {
            return true;
        }

        child = child->nextSibling;
    }

    return root->data == key;
}

bool findInOrder(Node* root, char key) {

    if (root == nullptr) {
        return false;
    }

    Node* child = root->firstChild;
```

```
    // First child
    if (child != nullptr) {
        if (findInOrder(child, key)) {
            return true;
        }
    }

    // Root
    if (root->data == key) {
        return true;
    }

    // Remaining children
    if (child != nullptr) {
        child = child->nextSibling;

        while (child != nullptr) {
            if (findInOrder(child, key)) {
                return true;
            }

            child = child->nextSibling;
        }
    }

    return false;
}

void findData(int choice, char key, Node* root) {
    bool found = false;

    if (choice == 1) {
        found = findPreOrder(root, key);
    }
    else if (choice == 2) {
        found = findPostOrder(root, key);
    }
    else if (choice == 3) {
        found = findInOrder(root, key);
    }

    if (found) {
        std::cout << key << " was found!" << std::endl;
    }
}

#endif
```

## Output

```
Find D using Pre-order: D was found!
Find D using Post-order: D was found!
Find D using In-order: D was found!
```

**Table 9-6**

### Main.cpp

```
Node* O = new Node('O');
addChild(G, O);
std::cout << "Find O using Pre-order: ";
findData(1, 'O', A);

    return 0;
```

### Output

```
Find O using Pre-order: O was found!
```

## Observation

**Adding node O child for node G extends the tree when calling the find data function it managed to successfully traverse down to node G and find the new child O thus printing the O was found demonstrating that functions like these automatically deals with newly added siblings.**

## 2. ANSWERS TO SUPPLEMENTAL ACTIVITY

**ILO C: Solve given problems using the tree data structure's implementation in C++**

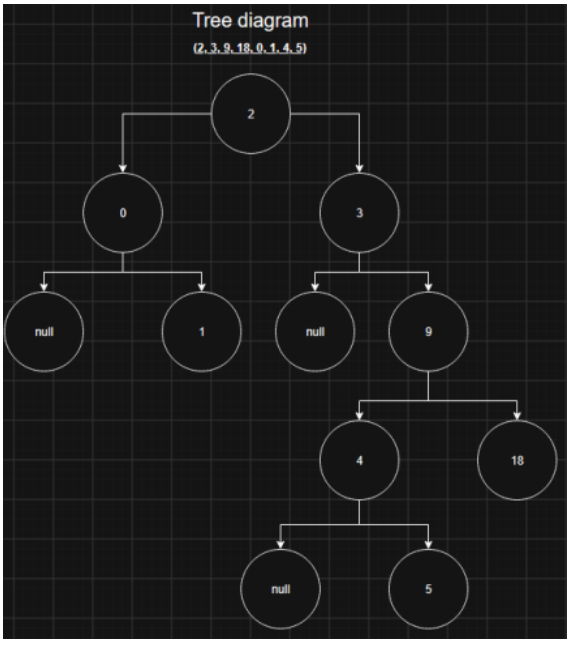
Step 1: Implement a binary search tree that will take the following values: 2, 3, 9, 18, 0, 1, 4, 5.

Main.cpp

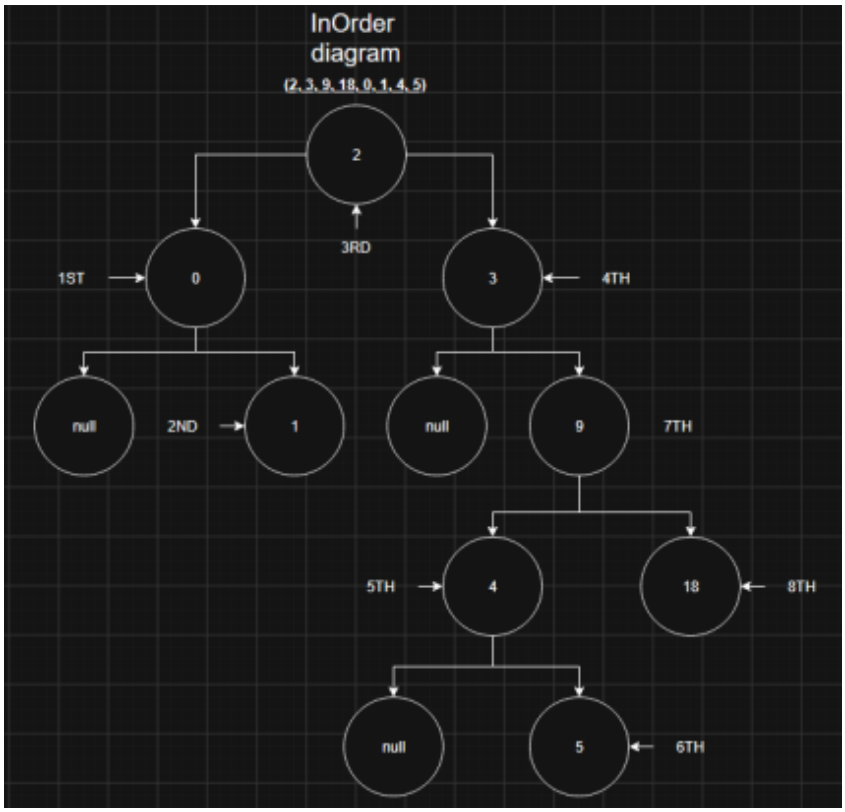
```
#include <iostream>
#include "BST.h"

int main(){
  //create the root
  BinarySearchTree<
//learn how to writ
//learn how to trav
  tree.insert(2); /
  tree.insert(3); /
  tree.insert(9);
  tree.insert(18);
  tree.insert(0);
  tree.insert(1);
  tree.insert(4);
  tree.insert(5);
```

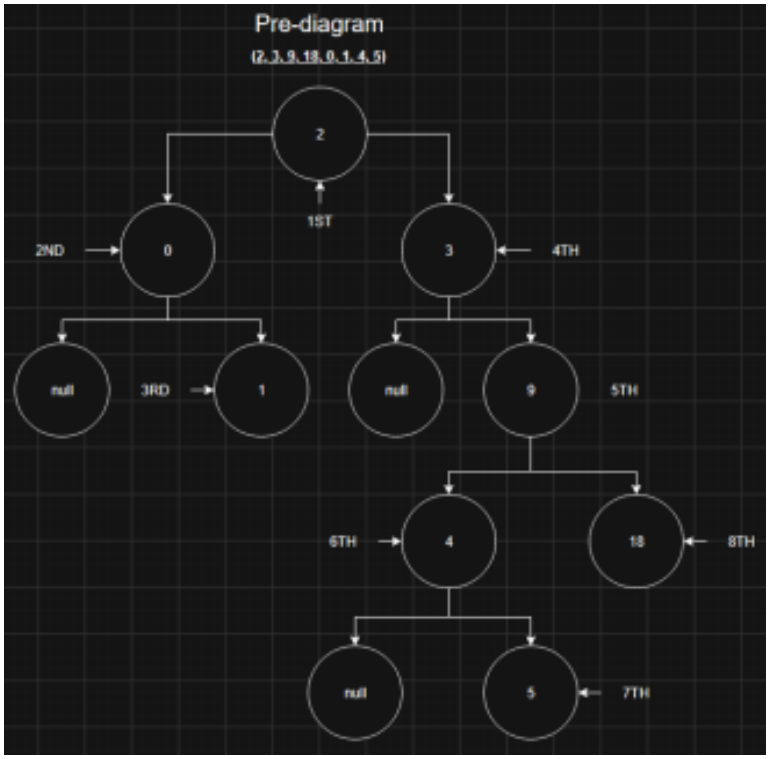
Step 2: Create a diagram to show the tree after all values have been inserted. Then, with the use of visual aids (like arrows and numbers) indicate the traversal order for in-order, pre-order and post-order traversal on the diagram.



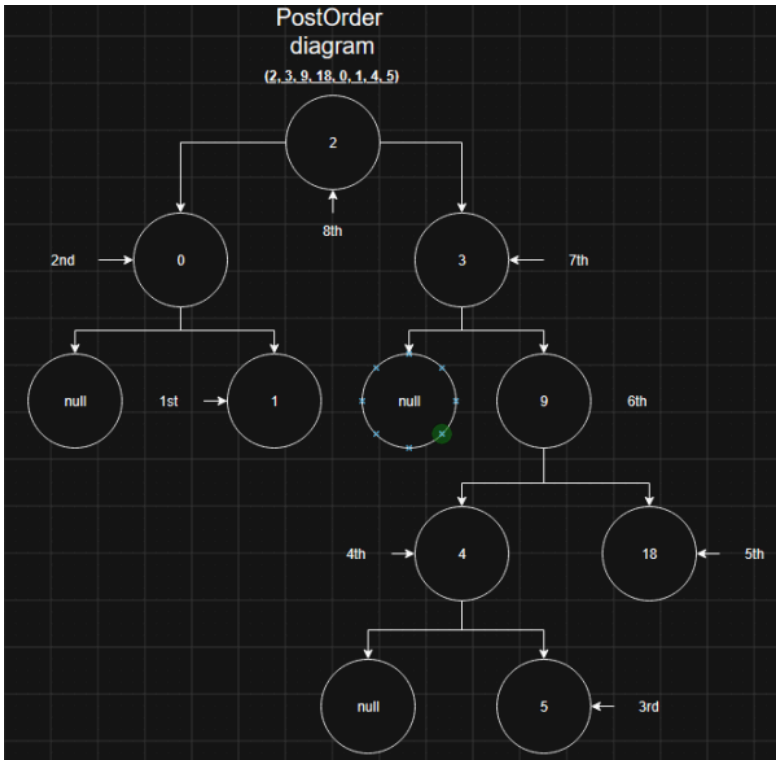
## IN ORDER TRAVERSAL




## PRE-ORDER TRAVERSAL



## POST-ORDER TRAVERSAL




Step 3: Compare the different traversal methods. In-order traversal was performed with what function?

```
PreOrder:  2 0 1 3 9 4 5 18
InOrder:   0 1 2 3 4 5 9 18
PostOrder: 1 0 5 4 18 9 3 2
```

Given the same input values above, what is the output with different traversal methods? For each output below, indicate your observation: is the output different from the pre-order and post-order traversal that you indicated in the diagrams shown in item #2.

**Pre-order Traversal**

```
template <typename T>
void BinarySearchTree<T>::preOrder() {
    std::cout << key << " ";
    if (left != nullptr) {
        left->preOrder();
    }
    if (right != nullptr) {
        right->preOrder();
    }
}
```

```
PreOrder:  2 0 1 3 9 4 5 18
```

**Post-order Traversal**

```
template<typename T>
void BinarySearchTree<T>::postOrder() {
    if (left != nullptr) {
        left->postOrder();
    }
    if (right != nullptr) {
        right->postOrder();
    }
    std::cout << key << " ";
}
```

```
PostOrder: 1 0 5 4 18 9 3 2
```

## CONCLUSION

**In this activity, I learned how to build and implement a binary tree using different root values and scenarios. I gained a clear understanding of how pre-order, in-order, and post-order traversals work to visit nodes in specific orders, and how to use these traversal methods to search for specific data within the tree structure.**

## 3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

**Tools Used:** **Chatgpt, Grammarly**

*Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.*

**Purpose of Use:** **Concept explanation, code assistance, grammar correction**

*Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.*

**Extent of Use:** **Explanation of concept, assistance of trouble shooting**

*Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.*

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.